PULSE AI

Security Self-Assessment

Implemented controls, partial safeguards, known design weaknesses and mitigation priorities.

Shenzhen International FinTech Competition 2026
International Track — Topic E: SME Finance Copilot

Evidence date: 20 October 2026. All example business records are synthetic. No performance or adoption metrics are assumed. Supabase/PostgreSQL is the team's shared deployment database; the evidence run uses isolated SQLite.

01 / Scope and threat model

This is an engineering self-assessment, not a security certification or penetration-test report. Scope: the inspected PULSE source, the synthetic backend/browser run, and observed local n8n configuration. Supabase is the selected shared PostgreSQL deployment, but its live connectivity and database policies were not verified in this local session.

Asset	Threat / trust boundary
Accounts and bearer tokens	Credential guessing, token theft through XSS, account takeover; browser-to-API boundary.
Tenant financial records	Cross-organization reads/writes, excessive role permissions; API-to-database boundary.
Financial decisions	Misleading forecasts, LLM unsupported claims, malicious imported data or prompt content.
Outbound workflows	Unauthenticated webhooks, duplicate dispatch, wrong recipients, forged success receipts.
Operational secrets	Leaked .env, signing keys, SMTP credentials, database URLs or n8n encryption keys.
Availability	Large uploads, unbounded forecasts, expensive parsing or provider delays.

Threat actors include unauthenticated callers, malicious tenant users, compromised browsers, adversarial content supplied to the copilot/importer, and operators with excessive database access. No claim is made that all attacks have been tested.

02 / Control status matrix

Control	Status	Evidence and boundary
JWT + bcrypt	IMPLEMENTED	Signed expiring bearer tokens; bcrypt hashes; no automatic refresh/revocation scheme proven.
Email OTP	IMPLEMENTED	HMAC hashes, expiry, attempts and resend cooldown; successful mailbox delivery separately confirmed in local work.
Account approval	IMPLEMENTED	Platform admin review; non-approved business requests blocked.
Tenant checks	IMPLEMENTED	Membership and role checks at application level; not Supabase RLS.
Tool allowlist	IMPLEMENTED	Registered tools and role-checked reminder proposals; no direct execution tool.
Output correctness	PARTIAL	Deterministic calculations and prompt grounding; LLM narrative not exhaustively verified.
Replay protection	PARTIAL	Status rejects sequential replay. Concurrent races and provider-side idempotency need work.
Auditability	PARTIAL	Database action/account logs exist. No append-only external storage or tamper-evidence.
Input limits	PARTIAL	Pydantic validation and import limits; full request/resource caps are incomplete.
Secret handling	PARTIAL	Local .env/runtime ignored; deployment secret manager/rotation not evidenced.
Global throttling / MFA	NOT IMPLEMENTED	No general login/IP rate limiter or MFA in inspected code.
Deployment assurance	NOT VERIFIED	No penetration test, Supabase RLS validation, backup restoration drill or dependency CVE audit performed.

03 / Permission model

Tier	Observed authority
Unauthenticated	Signup OTP/register/login endpoints; no business APIs.
Pending / rejected / suspended	Authenticated status visibility; business access denied.
VIEWER	Read tenant financial data and permitted analytical simulations.
ACCOUNTANT	Draft supported action proposals and import financial data, subject to route role checks.
FINANCE_MANAGER	Approve and execute supported actions; lower-role capabilities.
OWNER	Highest organization role; organization profile/management operations.
Platform administrator	Account approvals, organization metadata, system configuration summaries, account-review log. No automatic tenant finance bypass.

Role membership is checked server-side. Hiding a UI route is not the security boundary. The action and account-review tests exercise unauthorized access and state transitions.

04 / Authentication and secrets

Passwords are hashed with bcrypt. JWTs contain a subject and expiry and are signed using configured settings. Account active/approval state is re-read from the database, so a status suspension blocks existing tokens on subsequent API calls.

Email OTP is an email-possession check, not identity verification or MFA. Per-code attempt limits and per-address resend cooldown exist; IP-wide anti-abuse, distributed request throttling and enumeration resistance are incomplete. Email-existence responses can reveal whether an account is registered.

JWTs in localStorage are exposed if an attacker achieves XSS. There is no proven refresh-token family, central session revocation or stolen-token detection. Change signing keys and review session invalidation procedures before production.

Actual .env, credential files, runtime databases and n8n state are excluded from this package. A scanner compares publishable files against local SMTP/JWT/webhook secrets without printing them. .env.example is a placeholder template. Defaults and example secrets must never become production signing keys.

05 / Tenant isolation and Supabase

The primary isolation mechanism is require_membership plus require_role and organization-filtered queries. User-specific conversation access is also checked. Tests cover cross-organization access attempts. These are application controls and depend on every new endpoint applying them correctly.

Supabase/PostgreSQL is the team-selected shared database. The code does not establish Supabase Auth integration or a validated RLS policy set. A privileged database connection could bypass database-level isolation unless operationally restricted. Use a dedicated least-privilege application role, encrypted connections, network controls and reviewed policies.

Two laptops each using sqlite:///./pulse.db do not share registrations. The admin and every signup frontend must point to one shared backend/database deployment. The current local .env remained SQLite during capture; the evidence fixture was deliberately isolated. No claim of a passed Supabase connection or migration test is made.

06 / AI, prompt injection and financial risk

Unknown tools are denied. Tool inputs are tied to the authenticated organization, and reminder drafting enforces a minimum role. The orchestration loop is bounded in rounds, but not every model output is mechanically verified. A trace saying Risk Guardian review is not proof of a full semantic audit.

Treat user prompts, customer names, invoice descriptions, imported cells and retrieved provider messages as untrusted. Prompt instructions request grounded numbers, but an LLM can still produce unsupported language, select a poor scenario or misinterpret ambiguity. Do not describe prompt rules as an absolute hallucination blocker.

No RAG pipeline is implemented. Proposed future retrieval must enforce tenant scope before indexing/retrieval and label sources. Risks from cross-tenant retrieval are therefore roadmap concerns, not currently tested RAG controls.

Forecasts use assumptions and historical averages, not guarantees. Confidence labels are heuristic. Missing/duplicated data, invoice-versus-history overlap, month-end recurrences, inconsistent statuses and currencies can materially distort outputs. Add reconciliation, model validation and user-visible assumptions before production reliance.

07 / Actions, webhooks and audit

Approval and execution are separate endpoints. Execution requires FINANCE_MANAGER or OWNER, organization membership and APPROVED state. Sequential repeated execution is rejected. The synthetic demonstration proves the before-approval and replay denials without contacting a customer.

Known weakness: the current execution route reads status and dispatches before an atomic committed claim, so concurrent calls could race. A provider timeout may mean email was accepted but acknowledgment was lost; a fallback could duplicate delivery. Introduce atomic compare-and-set claims, durable outbox entries, provider idempotency keys and receipt reconciliation.

n8n webhooks use Header Auth and explicit receipts; production deployment needs HTTPS. Local HTTP is used only on loopback. Execution payload storage is disabled in the supplied local exports to avoid retaining OTP content. Any real mail recipient must be validated; a customer name is not an address.

Audit logs capture approval/execution/account-review events. Database operators can still modify tables. Stronger evidence requires restricted write privileges, append-only storage or signed external logs, retention policies and monitored access.

08 / Input validation and availability

Pydantic constrains registration fields and several scenario/import inputs. File import enforces a 5 MB check after reading the upload and a 5,000-row processing limit. This is not a streaming memory guard; compressed spreadsheets can consume more resources when expanded.

Forecast and some scenario horizon entry points need stricter bounds. Analytical simulation may be available to low-privilege members and still create persisted scenario records. Define quotas, request timeouts, bounded payloads and lifecycle cleanup.

Google Sheets fetching uses supported source parsing; redirects and allowlists should be reviewed for SSRF exposure. No live SSRF exploit or adversarial workbook test was performed. Do not claim these risks have been eliminated.

No comprehensive dependency CVE scan, external penetration test, load test, backup restore drill or incident-response exercise was performed. The test count demonstrates regression coverage, not proof of total security.

09 / Mitigation priorities

Priority	Action	Acceptance evidence
P0	Centralize backend on the shared Supabase deployment; use unique production secrets and TLS.	Verified shared signup/admin flow; migration log; secret/config review.
P0	Atomically claim actions and add durable idempotency/outbox processing.	Concurrent execution tests and timeout reconciliation without duplicate sends.
P0	Add IP/account rate limiting and strict compute/upload bounds.	Abuse tests and resource-budget checks.
P1	Harden browser sessions, CSP and XSS defenses; add MFA for admins.	Session threat review and browser security tests.
P1	Strengthen tenant database privileges/RLS where appropriate and audit new routes.	Database policy tests and endpoint isolation suite.
P1	Validate LLM claims and monetary/date outputs against tool evidence.	Adversarial prompt suite and structured response checks.
P2	Calibrate forecast uncertainty, improve reconciliation and operational monitoring.	Documented validation dataset, restore drills and alert exercises.

These are recommendations and planned work, not completed controls.

10 / Evidence and conclusion

The fresh regression run reports 86 tests passing. Actual financial checks, API responses, screenshot manifest and browser error results are included. The production frontend build succeeds with a large JavaScript chunk warning. The submission browser flow reports zero page errors on the exercised routes.

The no-key copilot path is verified here. Live LLM inference, Supabase policy behavior, security penetration testing and statistical forecast accuracy are not verified. Existing mocked provider tests must not be presented as an external model run.

PULSE is a functional prototype with meaningful permission, approval and audit controls. It should be presented as financial decision support with explicit assumptions, and hardened before handling production financial decisions at scale.